

STUDENT MANAGEMENT SYSTEM

Python Mini Project Report

Submitted By

Name: Rishika Gawande

Course: B.Tech

Branch: ECE with AI and Cybermatics

Language Used: Python

1. INTRODUCTION

The Student Management System is a menu-driven Python application developed to manage basic student records.

The main purpose of this project is to make student record management simple and organized. Instead of maintaining student information manually, the program allows the user to add, view, search, update and delete student records.

The system also calculates the grade of a student according to the marks entered. It provides branch-wise searching and basic statistics such as average marks, number of passed and failed students, and the topper.

This project is developed using basic Python concepts such as lists, dictionaries, functions, loops, conditional statements, input/output operations and basic data processing.

2. PROBLEM STATEMENT

Managing student information manually can become difficult when the number of students increases. Finding a particular student's details, updating marks, checking grades or identifying the topper may take unnecessary time.

The problem is to develop a simple Python-based system that can store and manage student information and perform common operations efficiently.

3.OBJECTIVES

The main objectives of the project are:

1. To create a simple system for managing student records.
2. To add new student information.
3. To prevent duplicate roll numbers.
4. To calculate grades automatically from marks.
5. To display all available student records.
6. To search for a student using roll number.
7. To search students according to their branch.
8. To update existing student information.

9. To delete student records.
10. To calculate basic student statistics.
11. To practice Python programming concepts through a practical application.

4. LANGUAGE AND CONCEPTS USED

Programming Language - Python

Concepts Used

- Variables
- Lists
- Dictionaries
- Functions
- "if-elif-else" statements
- "for" loops
- "while" loop
- User input
- Type conversion
- String methods
- List operations
- Basic mathematical calculations

Development Environment

The program can be executed using any Python-supported environment such as Python IDLE, VS Code, PyCharm or Pydroid 3.

5. DATA STRUCTURE USED

The project uses a list of dictionaries to store student records.

The main list is:

```
students = []
```

Each student is stored as a dictionary containing:

```
student = {  
    "roll": roll,  
    "name": name,  
    "branch": branch,  
    "marks": marks,  
    "grade": get_grade(marks)  
}
```

Therefore, one student record contains five pieces of information:

- Roll Number
- Name
- Branch

- Marks
- Grade

This structure makes it easier to access and update individual student details.

6. MAIN FEATURES

6.1 Add Student

The "add_student()" function allows the user to enter a new student's:

- Roll number
- Name
- Branch
- Marks

Before adding the student, the program checks whether the roll number already exists.

It also checks whether marks are between 0 and 100.

6.2 Automatic Grade Calculation

The "get_grade()" function calculates the grade based on marks.

Marks| Grade

90–100| A+

80–89| A

70–79| B

60–69| C

50–59| D

Below 50| F

This avoids manually entering grades and reduces the possibility of incorrect grade entries.

6.3 View Students

The "view_students()" function displays all stored student records.

If there are no records, the program displays:

No student records available.

Otherwise, the details of every student are displayed one by one.

6.4 Search Student

The "search_student()" function searches for a student using the roll number.

If the roll number is found, the program displays:

- Name
- Branch
- Marks
- Grade

If no matching roll number is found, it displays:

Student not found!

6.5 Search by Branch

The "search_by_branch()" function allows the user to find students belonging to a particular branch.

The program uses:

```
s["branch"].lower() == branch.lower()
```

This makes the comparison case-insensitive.

For example, entering:

ECE

or

ece

can match the same branch.

6.6 Update Student

The "update_student()" function allows existing student information to be changed.

The user can update:

- Name
- Branch
- Marks

After new marks are entered, the grade is recalculated automatically.

6.7 Delete Student

The "delete_student()" function removes a student record from the list using the student's roll number.

If the roll number exists, the corresponding record is deleted.

6.8 Student Statistics

The "show_statistics()" function calculates useful information from the stored records.

It displays:

- Total number of students
- Average marks
- Number of passed students
- Number of failed students
- Topper's name
- Topper's marks

The average is calculated using:

$$\text{average} = \text{total_marks} / \text{total}$$

The topper is identified by comparing the marks of all students.

7. FUNCTIONS USED

The project is divided into separate functions to make the program easier to understand and maintain.

Function	Purpose
"get_grade()"	Calculates student's grade
"add_student()"	Adds a new student
"view_students()"	Displays all students
"search_student()"	Searches using roll number
"search_by_branch()"	Searches students by branch
"update_student()"	Updates student information
"delete_student()"	Deletes a student record
"show_statistics()"	Displays student statistics

Using functions also follows a modular approach because each function performs a specific task.

8. PROGRAM WORKING

The program starts with an empty student list.

```
students = []
```

The main program then displays a menu containing eight options.

1. Add Student
2. View Students

3. Search Student
4. Search by Branch
5. Update Student
6. Delete Student
7. Student Statistics
8. Exit

The user enters a choice.

The program uses conditional statements to call the appropriate function.

For example:

```
if choice == "1":  
    add_student()  
elif choice == "2":  
    view_students()
```

The menu is inside a "while True" loop, so it continues to appear until the user selects option 8.

9. ALGORITHM

Step 1. Start the program.

Step 2. Create an empty list to store student records.

Step 3. Display the main menu.

Step 4. Take the user's choice.

Step 5. Perform the selected operation:

- Add student
- View students
- Search student
- Search by branch
- Update student
- Delete student
- Display statistics

Step 6. Return to the main menu.

Step 7. If the user selects Exit, terminate the program.

Step 8. End.

10. FLOWCHART OF THE PROGRAM

START

|

v

Create Empty List

|

v

Display Main Menu

|

v

Enter User Choice

|

+-----+-----+

| | |

v v v

Add Search Update

Student Student Student

| | |

+-----+-----+

|

v

Other Operations

|

v

Display Statistics

|

v

Return to Main Menu

|

Choice = 8?

/ \

No Yes

```
|      |  
v      v  
Menu    EXIT
```

11. INPUT VALIDATION

The project includes basic validation to prevent incorrect data.

Duplicate Roll Number

Before adding a student, the program checks whether the roll number already exists.

```
for s in students:
```

```
    if s["roll"] == roll:
```

```
        print("Roll number already exists!")
```

```
    return
```

Marks Validation

Marks must be between 0 and 100.

```
if marks < 0 or marks > 100:
```

```
    print("Marks should be between 0 and 100!")
```

```
    return
```

This prevents invalid marks from being stored.

12. IMPORTANT PYTHON CONCEPTS DEMONSTRATED

12.1 List

A list is used to store multiple student records.

```
students = []
```

12.2 Dictionary

A dictionary stores information about one student using key-value pairs.

```
{  
    "roll": roll,  
    "name": name,  
    "branch": branch,  
    "marks": marks,  
    "grade": grade  
}
```

12.3 Functions

Functions divide the program into smaller and manageable parts.

12.4 Loops

"for" loops are used to search and process student records.

The "while" loop keeps the main menu running.

12.5 Conditional Statements

"if", "elif" and "else" are used for decision-making.

12.6 String Methods

The ".lower()" method is used during branch searching to make the search case-insensitive.

13. SAMPLE OUTPUT

Main Menu

=====

STUDENT MANAGEMENT SYSTEM

=====

1. Add Student
2. View Students
3. Search Student
4. Search by Branch
5. Update Student

6. Delete Student

7. Student Statistics

8. Exit

=====

Enter your choice:

Adding a Student

Enter Roll No: 101

Enter Name: Riya

Enter Branch: ECE

Enter Marks: 87

Student added successfully!

Viewing Students

--- STUDENT RECORDS ---

Roll No : 101

Name : Riya

Branch : ECE

Marks : 87.0

Grade : A

Statistics

--- STUDENT STATISTICS ---

Total Students : 3

Average Marks : 78.67

Passed : 3

Failed : 0

Topper : Riya

Topper Marks : 87.0

14. TESTING

The program can be tested using different cases.

Test Case	Input/Condition	Expected Result
Add student	Valid details	Student added
Duplicate roll	Existing roll number	Error message
Invalid marks	Marks > 100	Invalid marks message
Invalid marks	Marks < 0	Invalid marks message
Search existing student	Valid roll number	Student details displayed
Search missing student	Unknown roll number	Student not found
Branch search	Existing branch	Matching students display
Update student	Existing roll number	Record updated
Delete student	Existing roll number	Record deleted
Statistics	Multiple records	Statistics displayed
Empty list	No records	No records message
Exit	Choice 8	Program terminates

15. ADVANTAGES

1. Simple and easy-to-use interface.
2. Reduces manual record management.
3. Automatically calculates grades.
4. Supports searching by roll number.
5. Supports branch-wise searching.
6. Allows updating and deleting records.
7. Provides useful student statistics.
8. Uses modular functions, making the code easier to maintain.
9. Demonstrates important Python programming concepts.

16. LIMITATIONS

The current version of the project has some limitations:

1. Student data is stored only while the program is running.
2. Records are lost when the program is closed.
3. There is no graphical user interface.
4. It does not use a database.
5. Authentication/login is not implemented.
6. The system currently focuses on basic academic information.

17. FUTURE SCOPE

The project can be improved further by adding:

- File handling for permanent data storage.
- SQLite or MySQL database.
- Login and authentication system.
- Graphical User Interface using Tkinter.
- Attendance management.
- Subject-wise marks.
- Automatic report card generation.
- Student performance charts.
- Exporting records to CSV or Excel.
- More detailed search and filtering options.

These additions can convert the current basic application into a more complete student management system.

18. CONCLUSION

The Student Management System is a practical Python project that demonstrates how programming concepts can be used to solve a real-world problem.

The project provides basic operations such as adding, viewing, searching, updating and deleting student records. It also includes automatic grade calculation and student performance statistics.

While developing this project, concepts such as functions, lists, dictionaries, loops, conditional statements, string methods and basic calculations were applied together in one application.

The project can be further developed by adding databases, file handling, authentication and a graphical interface. Therefore, it provides a good foundation for developing more advanced management applications in the future.

19. LEARNING OUTCOMES

After completing this project, I was able to understand and practice:

- How to divide a problem into smaller functions.
- How lists and dictionaries can store structured data.
- How loops can be used to search and process records.
- How conditional statements are used for decision-making.
- How to validate user input.
- How to perform calculations using stored data.
- How to design a menu-driven Python program.
- How different Python concepts can be combined to create a practical application.

20. PROJECT SUMMARY

Project Name: Student Management System

Programming Language: Python

Project Type: Console-based application

Main Data Structure: List of Dictionaries

Main Operations:

- Add
- View
- Search
- Search by Branch
- Update
- Delete
- Statistics
- Exit

Main Purpose:

To provide a simple and organized way to manage student records using Python.

DECLARATION

I hereby declare that this project titled “Student Management System” has been developed as a Python programming project for academic learning. The project demonstrates the practical implementation of fundamental programming concepts and problem-solving techniques.